Validation

son: 6 · FP Foundations · Category: Functional · Level: Intermediate · Build with Wa

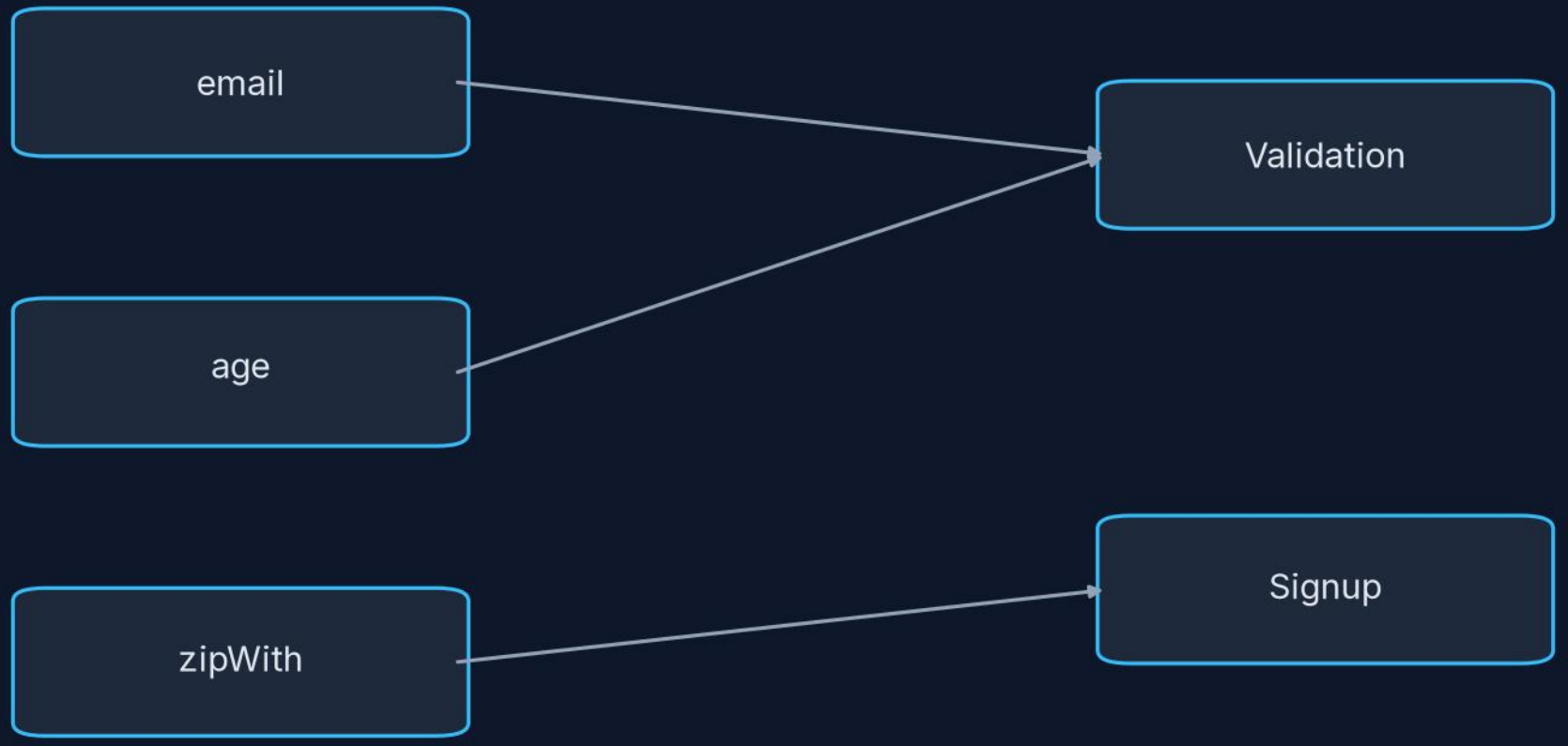
Today's Hook

- Today: Validation as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Validation structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Validation
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/functional/validation/SignupValidation.java
package com.buildwithwaqar.patterns.functional.validation;

import dev.zazr.control.Validation;

record Signup(String email, int age) {}

public final class SignupValidation {
    private SignupValidation() {}

    public static Validation<String, String> email(String raw) {
        return Validation.fromPredicate(raw, e -> e != null && e.contains("@"), e -> "email has no @");
    }

    public static Validation<String, Integer> age(int raw) {
        return Validation.fromPredicate(raw, a -> a >= 18, a -> "age under 18");
    }

    public static Validation<String, Signup> signup(String email, int age) {
        return Validation.zipWith(email(email), age(age), Signup::new);
    }
```

Implementation (2)

```
    public static String message(Validation<String, Signup> v) {
        return switch (v) {
            case Validation.Valid(var s) -> "welcome " + s.email();
            case Validation.Invalid(var errors) -> errors.mkString(", ");
        };
    }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.functional.validation;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Validation walkthrough ===");
    System.out.println(SignupValidation.message(SignupValidation.signup("a@b.c", 21)));
    System.out.println(SignupValidation.message(SignupValidation.signup("jules", -1)));
  }
}
```

Walkthrough Output

```
=== Validation walkthrough ===
welcome a@b.c
email has no @, age under 18
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Validation
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Validation → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.